

subject line. Subscription is done by telling a tuple daemon process on the same machine that monitors published tuples what subjects to look for.

Publish/subscribe is implemented as illustrated in Fig. 8-37. When a process has a tuple to publish, it broadcasts it out onto the local LAN. The tuple daemon on each machine copies all broadcasted tuples into its RAM. It then inspects the subject line to see which processes are interested in it, forwarding a copy to each one that is. Tuples can also be broadcast over a wide area network or the Internet by having one machine on each LAN act as an information router, collecting all published tuples and then forwarding them to other LANs for rebroadcasting. This forwarding can also be done intelligently, forwarding a tuple to a remote LAN only if that remote LAN has at least one subscriber who wants the tuple. Doing this requires having the information routers exchange information about subscribers.

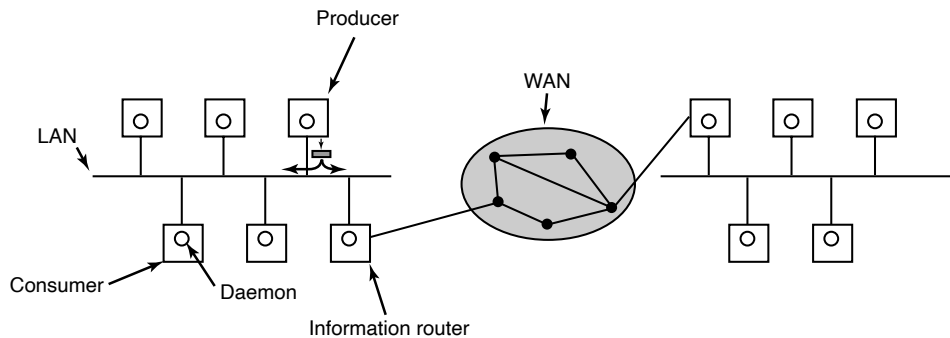


Figure 8-37. The publish/subscribe architecture.

Various kinds of semantics can be implemented, including reliable delivery and guaranteed delivery, even in the presence of crashes. In the latter case, it is necessary to store old tuples in case they are needed later. One way to store them is to hook up a database system to the system and have it subscribe to all tuples. This can be done by wrapping the database system in an adapter, to allow an existing database to work with the publish/subscribe model. As tuples come by, the adapter captures all of them and puts them in the database.

The publish/subscribe model fully decouples producers from consumers, as does Linda. However, sometimes it is useful to know who else is out there. This information can be acquired by publishing a tuple that basically asks: “Who out there is interested in x ?” Responses come back in the form of tuples that say: “I am interested in x .”

8.4 RESEARCH ON MULTIPLE PROCESSOR SYSTEMS

Research on multicores, multiprocessors, and distributed systems is extremely popular. Besides the direct problems of mapping operating system functionality on a system consisting of multiple processing cores, there are many open research

problems related to synchronization and consistency, and the way to make such systems faster and more reliable.

Thread management on cores remains an active and complex problem. To improve latency, Qin et al. (2020) implemented a user-level threads package that supports extremely short-lived threads (only a few microsecond). An arbiter core assigns cores to applications, and the applications then control the placements of threads among the cores.

Fast communication between nodes connected through a network, for instance, through RDMA, is also a hot research topic. Many optimizations for different (one-side or two-sided) RDMA primitives exist, and Wei et al. (2020) provide a systematic comparison. They show that no single primitive (one-sided or two-sided) wins in all cases and propose a hybrid implementation instead. Interestingly, the benefits of RDMA do not automatically apply to all programs written in languages such as Java and Scala, which do not support direct access to heap memory. Taranov et al. (2021) show how RDMA networking can be extended to Java.

The fast networks also lead to renewed interest in distributed shared memory (DSM). In DSM, caching of data (needed to reduce frequent remote accesses) can incur high coherence overhead. In Concordia, Wang et al. (2021) develop DSM with fast in-network cache coherence backed by smart NICs. Meanwhile, Ruan et al. (2020) demonstrated an application-integrated far memory implementation. It achieves the same common-case access latency for remote memory as for local RAM and allows one to build remoteable, hybrid near/far memory data structures.

While the design and implementation of new operating systems is getting rarer, even in research, new work does appear from time to time. LegoOS introduces a new OS model to manage disaggregated systems that disseminates traditional operating system functionalities into loosely coupled monitors, each of which runs on and manages a hardware component (Shan et al., 2018). Internally, LegoOS cleanly separates processor, memory, and storage devices both at the hardware level and the OS level.

One of the most difficult problems in distributed systems is what to do in case nodes fails. Alagappan et al. (2018) show how to perform replicated data updates in a distributed system using situation-aware updates and crash recovery. In particular, it will perform updates in memory if all is well and many nodes are up, but flushes them to disk when failures arise.

Finally, researchers work on using the abundance of many cores to improve storage. For instance, Liao et al. (2021) present a multicore-friendly log-structured file system (LFS) for flash storage. With three main techniques, they improve the scalability of LFS. First, they propose a new reader-writer semaphore to scale the user I/O without hurting the internal operations of LFS. Second, they improve the access to the in-memory index and cache while delivering a concurrency- and flash-friendly on-disk layout. Third, they exploit the flash parallelism, they move from a single log design with runtime-independent log partitions and delay the ordering and consistency guarantees to crash recovery.

8.5 SUMMARY

Computer systems can be made faster and more reliable by using multiple CPUs. Four organizations for multi-CPU systems are multiprocessors, multicomputers, virtual machines, and distributed systems. Each of these has its own properties and issues.

A multiprocessor consists of two or more CPUs that share a common RAM. Often these CPUs themselves consists of multiple cores. The cores and CPUs can be interconnected by a bus, a crossbar switch, or a multistage switching network. Various operating system configurations are possible, including giving each CPU its own operating system, having one leader operating system with the rest being follower, or having a symmetric multiprocessor, in which there is one copy of the operating system that any CPU can run. In the latter case, locks are needed to provide synchronization. When a lock is not available, a CPU can spin or do a context switch. Various scheduling algorithms are possible, including time sharing, space sharing, and gang scheduling.

Multicomputers also have two or more CPUs, but these CPUs each have their own private memory. They do not share any common RAM, so all communication uses message passing. In some cases, the network interface board has its own CPU, in which case the communication between the main CPU and the interface-board CPU has to be carefully organized to avoid race conditions. User-level communication on multicomputers often uses remote procedure calls, but distributed shared memory can also be used. Load balancing of processes is an issue here, and the various algorithms used for it include sender-initiated algorithms, receiver-initiated algorithms, and bidding algorithms.

Distributed systems are loosely coupled systems each of whose nodes is a complete computer with a complete set of peripherals and its own operating system. Often these systems are spread over a large geographical area. Middleware is often put on top of the operating system to provide a uniform layer for applications to interact with. The various kinds include document-based, file-based, object-based, and coordination-based middleware. Some examples are the World Wide Web, CORBA, and Linda.

PROBLEMS

1. What happens if two CPUs in a multiprocessor attempt to access exactly the same word of memory at exactly the same instant?
2. If a CPU issues one memory request every instruction and the computer runs at 200 MIPS, about how many CPUs will it take to saturate a 400-MHz bus? Assume that a memory reference requires one bus cycle. Now repeat this problem for a system in which caching is used and the caches have a 90% hit rate. Finally, what cache hit rate would be needed to allow 32 CPUs to share the bus without overloading it?